



ЗВІТ

до лабораторної роботи №8
Розширені можливості Redis

Виконала
Студентка 3 курсу
Групи МІТ-31
Факультету інформаційних технологій
Кубедінова Поліна

Тема: Розширені можливості Redis

Мета роботи: Закріпити знання про роботу з Redis та ознайомитися з розширеним функціоналом — транзакціями, скриптами на Lua, публікацією/підпискою (Pub/Sub) та потоками Redis Streams.

Хід роботи

1. Транзакції у Redis

Ознайомлення з базовими командами та WATCH

```
127.0.0.1:6379> SET balance 100
OK
127.0.0.1:6379> SET purchases 0
OK
127.0.0.1:6379>
127.0.0.1:6379> WATCH balance
OK
127.0.0.1:6379> MULTI
OK
127.0.0.1:6379(TX)> DECRBY balance 10
QUEUED
127.0.0.1:6379(TX)> INCR purchases
QUEUED
127.0.0.1:6379(TX)> EXEC
1) (integer) 90
2) (integer) 1
127.0.0.1:6379>
```

Проста транзакція, що додає кілька ключів

```
127.0.0.1:6379> MULTI
T user:1 "John"
SET user:2 "OK"
127.0.0.1:6379(TX)> SET user:1 "John"SET user:2 "Mary"
Invalid argument(s)
127.0.0.1:6379(TX)> SADD active:users "user:1" "user:2"
QUEUED
127.0.0.1:6379(TX)> EXEC
1) (integer) 2
127.0.0.1:6379>
```

Імітація конкурентного доступу (WATCH в дії)

Термінал 1

```
127.0.0.1:6379> WATCH balance
OK
127.0.0.1:6379> MULTI
OK
127.0.0.1:6379(TX)> DECRBY balance 20
QUEUED
127.0.0.1:6379(TX)>
```

Термінал 2 (Конкурент)

```
polin@Polin:/mnt/c/Users/polin$ redis-cli
127.0.0.1:6379> SET balance 150
OK
127.0.0.1:6379> |
```

Термінал 1

```
127.0.0.1:6379(TX)> EXEC
(nil)
127.0.0.1:6379>
```

2. Lua-скрипти в Redis

```
127.0.0.1:6379> EVAL "if redis.call('exists', KEYS[1]) == 0 then return redis.call('set', KEYS[1], ARGV[1]) else return 'exists' end" 1 testkey "my_value"
OK
127.0.0.1:6379>
```

3. Механізм Pub/Sub

Термінал 1 (Підписник / Subscriber)

```
OK
127.0.0.1:6379> SUBSCRIBE news
1) "subscribe"
2) "news"
3) (integer) 1
Reading messages... (press Ctrl-C to quit or any key to type command)
```

Термінал 2 (Видавець / Publisher)

```
OK
127.0.0.1:6379> PUBLISH news "Redis is awesome!"
(integer) 1
127.0.0.1:6379> |
```

Результат у Терміналі 1

```
OK
127.0.0.1:6379> SUBSCRIBE news
1) "subscribe"
2) "news"
3) (integer) 1
1) "message"
2) "news"
3) "Redis is awesome!"
Reading messages... (press Ctrl-C to quit or any key to type command)
```

4. Redis Streams (поток даних)

Додавання подій до потоку

```
127.0.0.1:6379> XADD mystream * sensor-id 1234 temperature 19.8
sensor-id 1234 temperature 20.3"1778941758816-0"
127.0.0.1:6379> XADD mystream * sensor-id 1234 temperature 20.3
"1778941760081-0"
127.0.0.1:6379>
```

Читання подій з потоку

```
127.0.0.1:6379> XRANGE mystream - +
1) 1) "1778941758816-0"
   2) 1) "sensor-id"
      2) "1234"
      3) "temperature"
      4) "19.8"
2) 1) "1778941760081-0"
   2) 1) "sensor-id"
      2) "1234"
      3) "temperature"
      4) "20.3"
127.0.0.1:6379>
```

Зчитування нових повідомлень за допомогою XREAD

```
127.0.0.1:6379> XREAD COUNT 2 STREAMS mystream 0
1) 1) "mystream"
   2) 1) 1) "1778941758816-0"
      2) 1) "sensor-id"
         2) "1234"
         3) "temperature"
         4) "19.8"
      2) 1) "1778941760081-0"
         2) 1) "sensor-id"
            2) "1234"
            3) "temperature"
            4) "20.3"
127.0.0.1:6379>
```

5. (Додатково) Напишіть скрипт (Python/JS), який реалізує логіку Pub/Sub або читає дані зі Stream.

```
import redis
import time

def main():
    print("=== Лабораторна робота №8: Робота з Redis Streams через Python ===")

    try:
        r = redis.Redis(host='localhost', port=6379,
decode_responses=True)
        if r.ping():
            print("[OK] Підключення до Redis успішне!")
    except redis.ConnectionError:
```

```

        print("[ПОМИЛКА] Не вдалося підключитися. Запустіть сервер:
sudo service redis-server start")
        return

    stream_name = 'weather_stream'

    r.delete(stream_name)

    print(f"\n1. Додавання нових подій до потоку
'{stream_name}'...")

    data_points = [
        {"sensor_id": "WS-01", "temp": "18.5", "humidity": "65%"},
        {"sensor_id": "WS-01", "temp": "19.0", "humidity": "64%"},
        {"sensor_id": "WS-02", "temp": "21.2", "humidity": "50%"}
    ]

    for data in data_points:

        event_id = r.xadd(stream_name, data, id=('*'))
        print(f" -> Додано подію з ID: {event_id} | Дані: {data}")
        time.sleep(0.5)

    print(f"\n2. Читання всіх подій з потоку '{stream_name}' за
допомогою XRANGE...")

    events = r.xrange(stream_name, min='-', max='+')
    for event in events:
        event_id, event_data = event
        print(f" -> [{event_id}] Сенсор:
{event_data.get('sensor_id')}, "
              f"Температура: {event_data.get('temp')}°C, Вологість:
{event_data.get('humidity')}")

    print(f"\n3. Читання нових подій за допомогою XREAD...")

    read_data = r.xread({stream_name: '0-0'}, count=2)
    for stream, messages in read_data:
        print(f" -> Зчитано {len(messages)} повідомлень з потоку
'{stream}':")
        for msg_id, msg_data in messages:
            print(f"      - ID: {msg_id} -> {msg_data}")

    print("\n=== Тестування програми на Python успішно завершено
===")

```

```
if __name__ == "__main__":  
    main()
```

КОД

```
D:\polin\University>python -u "d:\polin\University\Databases_Kubiedinova\tempCodeRunnerFile.py"  
=== Лабораторна робота №8: Робота з Redis Streams через Python ===  
[OK] Підключення до Redis успішне!  
  
1. Додавання нових подій до потоку 'weather_stream'...  
-> Додано подію з ID: 1778941964228-0 | Дані: {'sensor_id': 'WS-01', 'temp': '18.5', 'humidity': '65%'}  
-> Додано подію з ID: 1778941964730-0 | Дані: {'sensor_id': 'WS-01', 'temp': '19.0', 'humidity': '64%'}  
-> Додано подію з ID: 1778941965231-0 | Дані: {'sensor_id': 'WS-02', 'temp': '21.2', 'humidity': '50%'}  
  
2. Читання всіх подій з потоку 'weather_stream' за допомогою XRANGE...  
-> [1778941964228-0] Сенсор: WS-01, Температура: 18.5°C, Вологість: 65%  
-> [1778941964730-0] Сенсор: WS-01, Температура: 19.0°C, Вологість: 64%  
-> [1778941965231-0] Сенсор: WS-02, Температура: 21.2°C, Вологість: 50%  
  
3. Читання нових подій за допомогою XREAD...  
-> Зчитано 2 повідомлень з потоку 'weather_stream':  
  - ID: 1778941964228-0 -> {'sensor_id': 'WS-01', 'temp': '18.5', 'humidity': '65%'}  
  - ID: 1778941964730-0 -> {'sensor_id': 'WS-01', 'temp': '19.0', 'humidity': '64%'}  
  
=== Тестування програми на Python успішно завершено ===  
  
D:\polin\University>
```

виконання

6. Теоретичне завдання

- a. Визначте, які з нових механізмів Redis можуть бути корисними у великих розподілених системах.

У великих розподілених системах надзвичайно корисним є механізм **Redis Streams**, оскільки він дозволяє створювати стійкі черги повідомлень та шини даних для асинхронної взаємодії між мікросервісами. Також важливу роль відіграють **Lua-скрипти**, які забезпечують атомарне виконання складної логіки безпосередньо на сервері, мінімізуючи мережеві затримки та навантаження на систему. Окрім цього, використання **транзакцій** разом із командою **WATCH** дозволяє підтримувати цілісність даних при одночасному доступі багатьох користувачів до одного ресурсу. Всі ці інструменти в сукупності роблять систему більш масштабованою та надійною в умовах високих навантажень.

- b. Які плюси і мінуси Redis як брокера повідомлень?

Головним плюсом Redis є його екстремальна швидкість роботи через зберігання даних в оперативній пам'яті, що забезпечує мінімальні затримки при доставці повідомлень. Крім того, він дуже гнучкий, оскільки підтримує різні моделі взаємодії: від простої розсилки через Pub/Sub до стійких черг із гарантією доставки за допомогою Redis Streams. Однак суттєвим мінусом є ризик втрати даних при раптовому вимкненні сервера,

оскільки повідомлення за замовчуванням зберігаються в RAM, а не на диску. Також у порівнянні зі спеціалізованими брокерами, такими як RabbitMQ, Redis має обмежені можливості для складної маршрутизації повідомлень.

Висновок

Під час виконання лабораторної роботи №8 було детально вивчено та практично застосовано розширений функціонал системи керування базами даних Redis. Особливу увагу було приділено механізму транзакцій та команді **WATCH**, що дозволило імітувати конкурентний доступ до даних та забезпечити їхню цілісність. Використання Lua-скриптів продемонструвало можливість перенесення складної логіки на сторону сервера для забезпечення атомарності операцій. Практичне освоєння моделей Pub/Sub та Redis Streams дало розуміння того, як організувати ефективну передачу повідомлень та обробку потоків подій у реальному часі. Розробка Python-скрипту закріпила навички програмної взаємодії з асинхронними структурами даних Redis. У результаті було зроблено висновок, що ці механізми є незамінними для побудови масштабованих розподілених систем, де швидкість та надійність обміну даними є критично важливими.